

Project Report — Search Typeahead System

Author: Saniya Patil **Course:** HLD101 — High Level Design **Repository:**
<https://github.com/Saniya1613/search-typeahead>

1. Overview

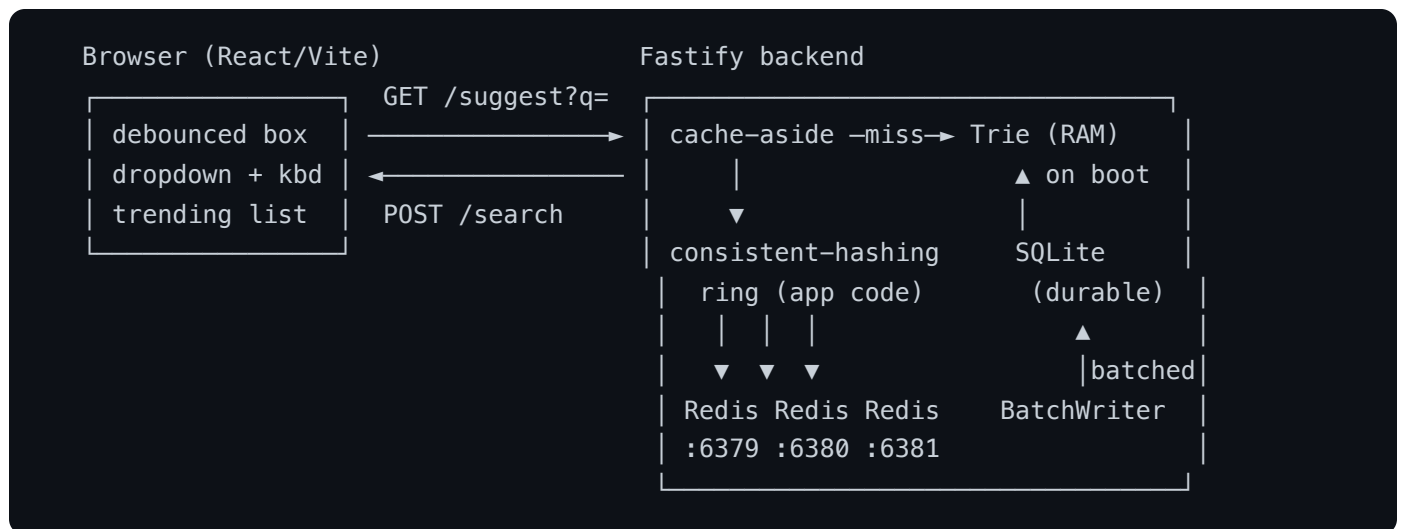
A search typeahead (autocomplete) system that returns popularity-ranked prefix suggestions as the user types, records committed searches to update popularity, serves repeated lookups from a **distributed cache addressed by consistent hashing**, persists search counts using **batched writes**, and ranks **trending searches** by recent activity rather than all-time popularity. A small React frontend exercises the full flow.

The design separates two concerns that are often conflated: **durability** (a SQLite source of truth) and **read speed** (an in-memory trie index rebuilt from SQLite on boot). Everything else — cache, batching, trending — is built around that split.

2. Tech stack

Layer	Technology	Reason
Backend	Node.js + TypeScript, Fastify	Typed, fast, minimal boilerplate
Primary store	SQLite (<code>better-sqlite3</code>)	Durable, zero-ops, inspectable
Read index	Hand-written Trie	O(prefix length) lookups
Cache	3 Redis nodes + hand-written consistent-hashing ring (<code>ioredis</code>)	Distributed cache graded by the rubric
Frontend	React 18 + Vite	Debounce, keyboard nav, trending UI
Dataset	Python <code>wordfreq</code> (149,998 rows)	Real frequency signal, > 100k minimum
Infra	Docker Compose (3 Redis containers)	“Use Docker” requirement

3. Architecture



Request flow (typing “ip”): debounced `GET /suggest?q=ip` → hash key `suggest:ip` → ring picks the owning Redis node → hit returns cached top-10; miss walks the trie, sorts by count, caches with a 60 s TTL, returns.

Search flow (Enter): `POST /search` → returns `{message:"Searched"}`, bumps the in-memory trie immediately, enqueues the event; the batch flusher writes the aggregated counts to SQLite and invalidates affected cache keys.

4. Key design decisions and trade-offs

Decision	Why	Trade-off accepted
Trie as read index, not SQL <code>LIKE</code>	O(P) lookup, no per keystroke table scan	Trie lives in RAM, ~300 ms to rebuild on boot
<code>Map</code> children in trie	Supports digits/apostrophes, not just a–z	Slightly more memory than a fixed array
Consistent hashing, not <code>hash % N</code>	Adding/removing a node moves only ~K/N keys, not ~all	More code than a modulus
150 virtual nodes/node	Even load (~1/3 each); measured 645/615/617	More ring entries to sort/search
FNV-1a + <code>fmix32</code> finalizer	Good avalanche → even distribution (43% → 9% spread)	—
Cache-aside, 60 s TTL	Simple, cache never a correctness dependency	Up to 60 s staleness (bounded; + explicit invalidation)
Batched writes (size 500 / 2 s)	~99% fewer DB writes under load	Un-flushed events lost on crash (≤ ~2 s)

Decision	Why	Trade-off accepted
Recency = exp. decay over hourly buckets	One-time spikes decay away naturally	Window/half-life is a freshness↔stability tuning choice
$\alpha \cdot \ln(1+\text{count}) + \beta \cdot \text{recency}$ blend	Popularity prior without letting giants dominate	Two constants to justify ($\alpha=1$, $\beta=2$)

Each of these is explained inline in the source code comments, which document the reasoning behind every component.

5. Data model

- `queries(query PRIMARY KEY, count)` — all-time popularity (source of truth).
- `query_hourly(query, hour, hits)` — recent activity bucketed by clock-hour, feeds the trending recency score; pruned to a 48 h window.
- SQLite runs in WAL mode so committed writes are crash-safe.

6. Performance (measured)

`npm run bench` — 5000 serial `/suggest` requests, hot-prefix-biased workload, backend + 3 Redis nodes on one laptop:

Metric	Value
Latency p50	~0.3 ms
Latency p95	~2 ms
Latency p99	~8 ms
Cache hit rate	~75%
Throughput (1 serial client)	~1,100 req/s
Trie build on boot	~300 ms / 150k rows
Batch write reduction	1000 events → 7 DB writes (99.3%)

Live counters are exposed at `GET /metrics`.

7. Requirement coverage

#	Requirement	Status	Evidence
1	Prefix suggestions, ≤10, sorted by count desc	✓	<code>trie.ts</code> , <code>routes/suggest.ts</code>
2	Empty / no-match / mixed-case handled	✓	<code>Trie.normalize</code> , <code>suggest route</code>
3	Debounced UI, avoids redundant calls	✓	<code>App.jsx</code> (300 ms)
4	GET <code>/suggest?q=<prefix></code>	✓	<code>routes/suggest.ts</code>
5	POST <code>/search</code> → <code>{message:"Searched"}</code>	✓	<code>routes/search.ts</code>
6	Existing query count++, new query created	✓	batch upsert
7	Durable query-count store	✓	SQLite
8	Cache checked before DB; stores prefix results	✓	cache-aside in suggest
9	Cache expiry / invalidation (not stale forever)	✓	60 s TTL + flush-time invalidation
10	Cache distributed across multiple nodes	✓	3 Redis nodes
11	Consistent hashing decides key ownership	✓	<code>cache/ring.ts</code>
12	GET <code>/cache/debug?prefix=</code> (node + hit/miss)	✓	<code>routes/cache.ts</code>
13	Trending — basic (count)	✓	<code>/trending?mode=basic</code>
14	Trending — enhanced (recency)	✓	<code>/trending?mode=enhanced</code>
15	Recency tracking + decay + rank-change invalidation	✓	<code>trending.ts</code> , batch invalidation
16	Batch writes: buffer, aggregate, size/time flush	✓	<code>batch.ts</code>
17	Show DB-write reduction	✓	<code>/metrics</code> (99%)

#	Requirement	Status	Evidence
18	Crash-before-flush discussed	✓	this report §4; <code>batch.ts</code> comments
19	UI: box, dropdown, enter/click, dummy response, trending, loading/error, keyboard	✓	<code>App.jsx</code>
20	Non-functional: runs locally, p95 measured, hit rate, modular, documented	✓	<code>bench.ts</code> , <code>/metrics</code> , docs
21	Docker	✓	<code>docker-compose.yml</code>

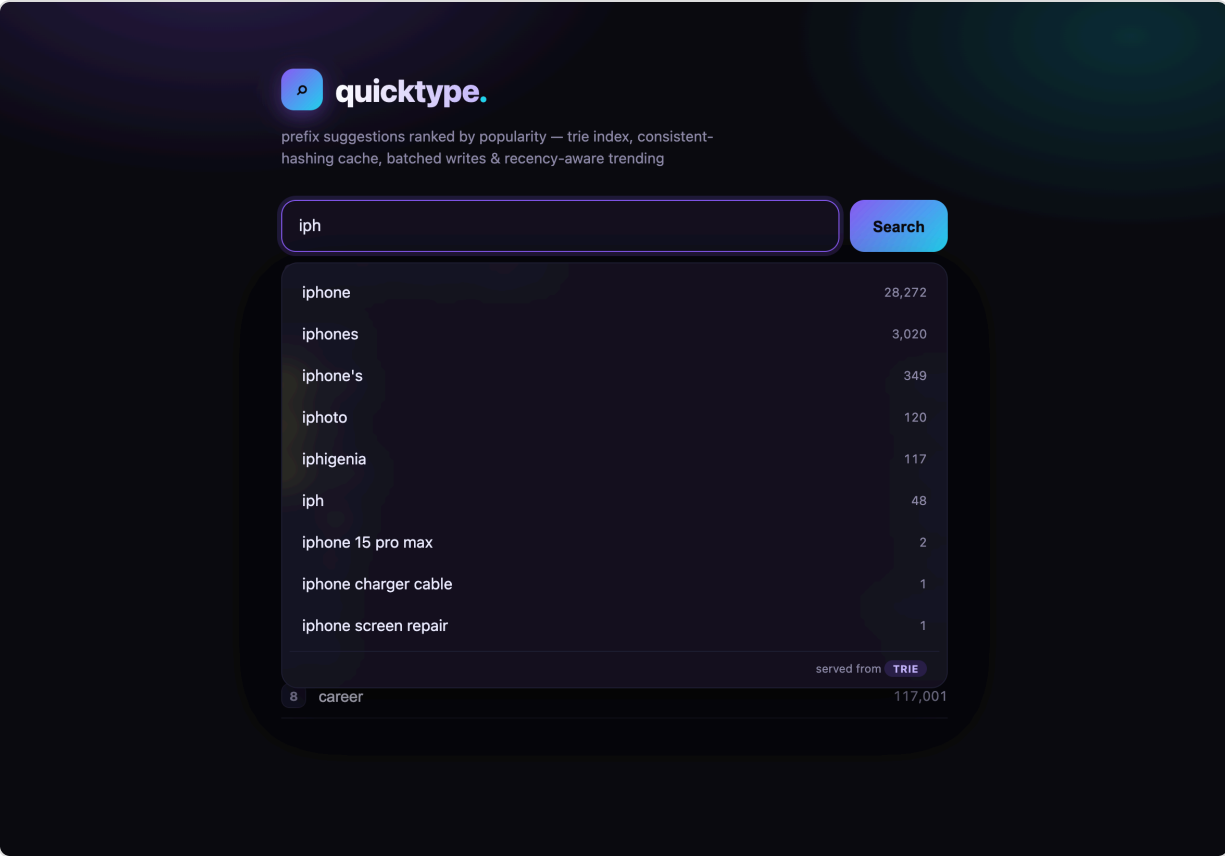
Known limitations and future work: - Suggestions for very short prefixes (1 char) do a collect+sort; a production system would precompute top-k per trie node. Mitigated by cache + measured p95. - Batch buffer is in-process, so a crash loses $\leq \sim 2$ s of count increments; a production system would use a durable queue. - DB *read* counts aren't separately metered (reads are almost entirely the in-memory trie; SQLite reads happen on boot and for trending only).

8. How to run

See the README “Running it” section. In short: generate + ingest the dataset, start the 3 cache nodes (`docker compose up -d` or `npm run redis:up`), then `npm run dev` in `backend/` and `frontend/`.

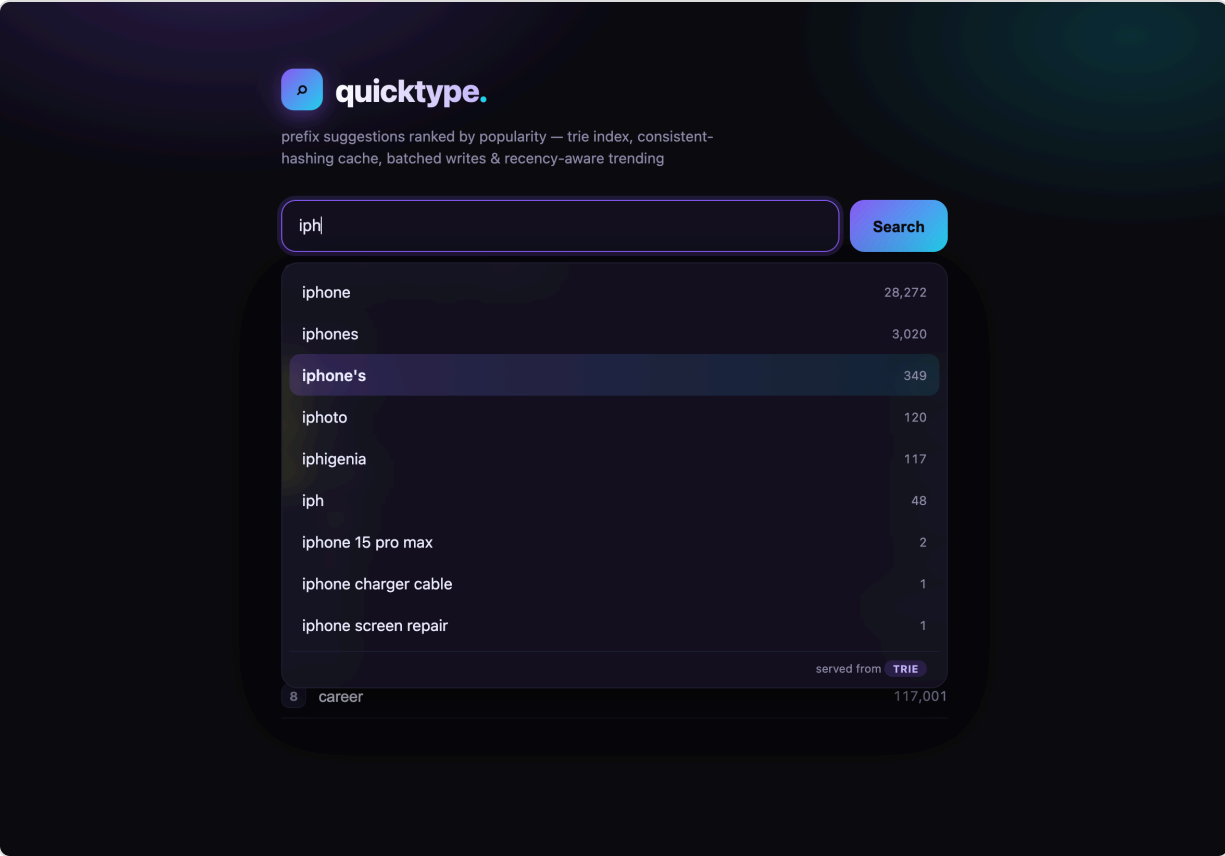
9. Screenshots

Prefix suggestions (typeahead), ranked by popularity:



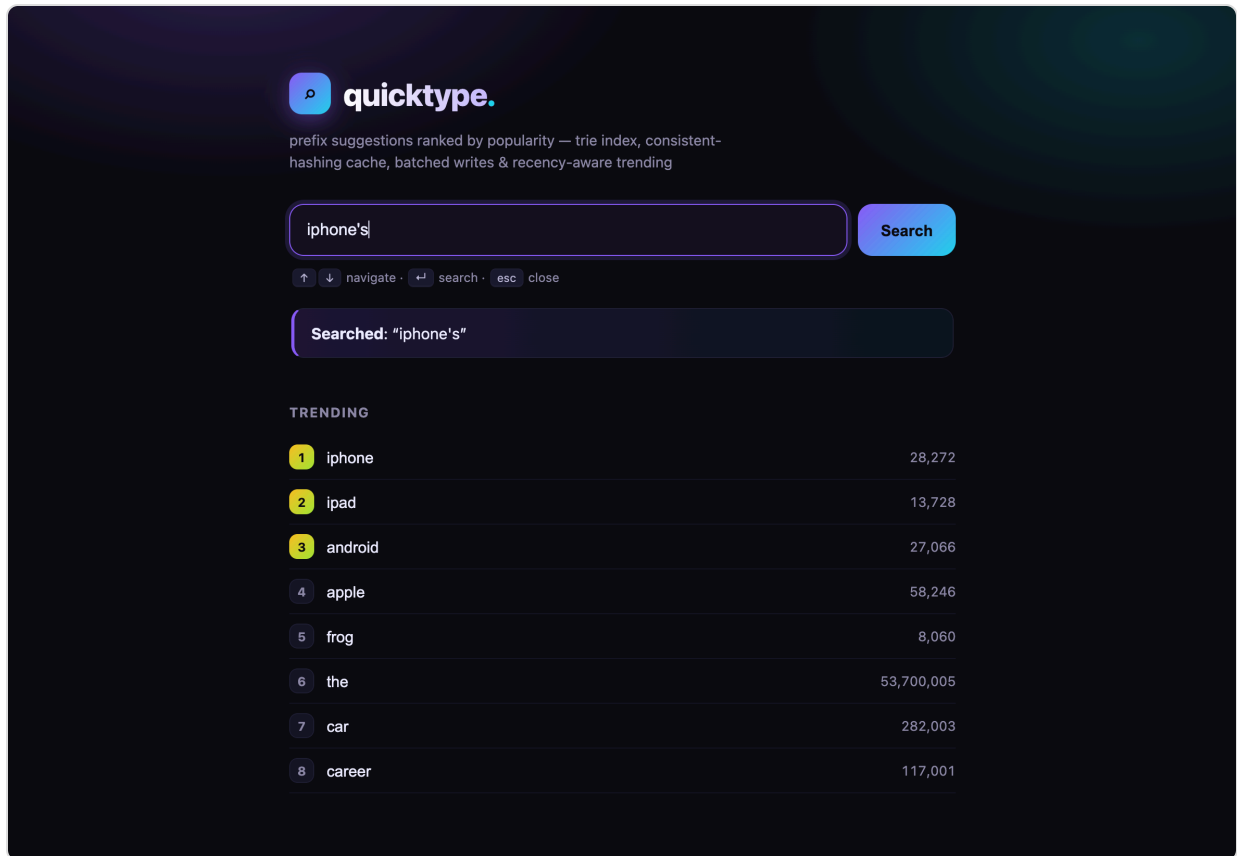
Suggestions dropdown

Keyboard navigation through the dropdown:



Keyboard navigation

Search submitted — dummy response + trending list:



Search result and trending

Backend: consistent-hashing load balance, cache routing, and batch metrics:

```
Backend API - consistent hashing, cache, batch metrics

$ curl -s localhost:3001/cache/ring # consistent-hashing balance over 3 nodes
{
  "totalKeys": 1877,
  "perNode": {
    "cache-a": 645,
    "cache-b": 615,
    "cache-c": 617
  }
}

$ curl -s 'localhost:3001/cache/debug?prefix=iphone' # node ownership + hit/miss
{
  "prefix": "iphone",
  "key": "suggest:iphone",
  "node": "cache-c",
  "host": "127.0.0.1",
  "port": 6381,
  "hit": false
}

$ curl -s localhost:3001/metrics # cache hit rate + batch write reduction
{
  "cache": {
    "hits": 3903,
    "misses": 1302,
    "hitRate": 0.75
  },
  "batch": {
    "enqueued": 3013,
    "flushes": 8,
    "dbWrites": 31,
    "pending": 0,
    "lastFlushSize": 4,
    "writeReduction": 0.99
  },
  "queries": 149999
}
```

Backend metrics

Latency benchmark (p50/p95/p99) and cache hit rate:

```
npm run bench -- /suggest latency + cache hit rate
```

```
=== npm run bench ===  
=== /suggest benchmark (5000 requests) ===  
throughput   : 3552 req/s (single client, serial)  
latency p50  : 0.21 ms  
latency p95  : 0.58 ms  
latency p99  : 1.26 ms  
latency max  : 10.89 ms  
cache hits   : 3903  
cache misses : 1302  
cache hitRate: 75.0%
```

Benchmark